



The Complete Guide

A programming language that reads like group chat
and runs like Python.

Everything you need to go from your first line
to building whatever you like.

Version 1.1.0 · 03 September 2026

Every example in this guide was run by the W++ interpreter while the guide was being written.

Contents

1. What W++ is
2. Getting set up
3. The Official Dictionary
4. Printing with yap
5. Reading input with dm
6. Variables and values
7. Numbers and arithmetic
8. Text
9. Yes, no and nothing
10. Making decisions
11. Repeating things
12. Collections
13. Building collections in one line
14. Functions
15. When things go wrong
16. Reserved words: the one real gotcha
17. Everything Python brings with it
18. Tools that help
19. Complete programs
20. Habits worth picking up
21. What W++ does not do
22. Where to go next
23. Cheat sheet

1. What W++ is

W++ is a programming language that reads like group chat and runs like Python. Every W++ keyword is a nickname for a Python one: **cook** means `def`, **yap** means `print`, **bet** means `if`. There are nineteen of these nicknames and that is the whole language.

This matters more than it sounds, and it is the single most useful thing to understand as a beginner: **anything Python can do, W++ can do**. Classes, generators, decorators, the entire standard library, f-strings, the walrus operator - all of it works, unchanged. You are not learning a small toy language with a short list of features. You are learning Python with a different set of words for nineteen things.

How your program actually runs

When you run a `.wpp` file, four things happen in order:

```
your_program.wpp
|
| 1. read the file
v
swap the keywords      cook -> def, yap -> print, ...
|
| 2. this is the only translation step
v
ordinary Python source  see it yourself with --emit
|
| 3. hand it to Python
v
your output, or a Skill Issue
```

Two rules keep step 2 honest, and they are worth knowing early because they explain nearly every surprise:

- **Keywords are matched as whole words.** A variable called `cookie` is left alone; only a standalone `cook` becomes `def`.
- **Text inside strings and comments is never translated.** `yap("cook dinner")` prints *cook dinner*, not *def dinner*.

Line numbers are preserved exactly, so when something goes wrong the error points at the line you wrote, not at generated code you have never seen.

2. Getting set up

You need **Python 3.8 or newer** and nothing else. W++ has no dependencies to install.

```
$ git clone https://github.com/harrynoble/wplusplus.git
$ cd wplusplus
```

Check that it works:

```
$ python wpp.py examples/hello.wpp
```

```
Hello world
```

If you see *Hello world*, you are done. There is nothing to build and nothing to configure.

The two ways to run W++

1. From the terminal. Put your program in a file ending in `.wpp` and run it:

```
$ python wpp.py my_program.wpp
```

2. In the playground. A local web page with a code editor and a terminal-style output panel, which is the friendlier place to start:

```
$ python playground/server.py
```

That opens `http://127.0.0.1:8000` in your browser. Pick something from the **Examples** menu, press **Run** (or Ctrl+Enter), and the output appears beside your code. When a program asks you a question, you type your answer straight into the output panel.

The playground runs the code you give it on your own machine. That is fine for learning, but do not expose it to a network.

Your first program

Create a file called `hello.wpp` containing one line:

```
yap("Hello world")
```

```
Hello world
```

Then run it with `python wpp.py hello.wpp`. That is a complete W++ program - there is no boilerplate, no main function, no imports needed.

3. The Official Dictionary

These nineteen words are the entire language. Everything else you write in W++ is ordinary Python. Learn this table and you have learned W++; the rest of this guide is about what to do with it.

W++	Python	What it means
cook	def	start a function
spill	return	send a value back
yap	print	print to the screen
dm	input	ask the user for input
bodycount	len	how many items
bet	if	if this is true
plotwist	elif	otherwise, if this is true
nah	else	otherwise
spam	for	repeat for each item
grind	while	repeat while true
dip	break	leave the loop now
skrrt	continue	skip to the next round
nocap	True	true
cap	False	false
npc	None	nothing / no value
squad	list	a list
tea	dict	a dictionary
cult	set	a set
range	range	a run of numbers

These nineteen words are **reserved**. You cannot use one as the name of your own variable, function or class - see section 16, which explains the one error every beginner hits.

4. Printing with yap

yap puts things on the screen. Give it as many values as you like and it separates them with a space:

```
yap("Hello world")
yap("one", 2, nocap)
yap()
yap("three", "values", "again")
```

```
Hello world
one 2 True

three values again
```

An empty **yap()** prints a blank line. Two options let you control the spacing and the line ending:

```
yap("a", "b", "c", sep="-")
yap("no line break yet", end=" ... ")
yap("continued on the same line")
```

```
a-b-c
no line break yet ... continued on the same line
```

Anything can be printed, not just text:

```
yap(42, 3.5, nocap, npc)
yap(squad([1, 2, 3]))
yap(tea(name="wpp", fun=nocap))
```

```
42 3.5 True None
[1, 2, 3]
{'name': 'wpp', 'fun': True}
```

Notice that **nocap** printed as *True* and **npc** printed as *None*. The keywords are how you *write* the values; by the time the program runs it really is Python, so the values print the way Python prints them.

5. Reading input with dm

dm asks the person running your program for a line of text. Whatever you pass to it is shown as the prompt:

```
name = dm("What is your name? ")
yap("Nice to meet you,", name)
```

```
What is your name? Ada
Nice to meet you, Ada
```

In the guide above, *Ada* is what was typed at the prompt. In the playground the cursor appears right after the question in the output panel; in the terminal you type and press Enter as usual.

dm always gives you text, even when the user types digits. To do maths with it, convert it first:

```
raw = dm("Pick a number: ")
n = int(raw)
yap("Doubled:", n * 2)
yap("But without converting:", raw * 2)
```

```
Pick a number: 7
Doubled: 14
But without converting: 77
```

Use `int()` for whole numbers and `float()` for decimals. If the user types something that is not a number, the conversion fails - section 15 shows how to handle that gracefully.

You can ask more than once, including inside a loop:

```
total = 0
spam i in range(3):
    total = total + int(dm("Score " + str(i + 1) + ": "))
yap("Total:", total)
```

```
Score 1: 10
Score 2: 20
Score 3: 30
Total: 60
```

6. Variables and values

A variable is a name for a value. You make one by assigning to it, and you never declare a type:

```
score = 10
name = "Ada"
ready = nocap

yap(score, name, ready)

score = score + 5
yap("after a change:", score)

score = "now a string"
yap("a name can hold anything:", score)
```

```
10 Ada True
after a change: 15
a name can hold anything: now a string
```

You can assign several names at once, which makes swapping two values a one-liner:

```
a, b = 1, 2
yap(a, b)

a, b = b, a
yap("swapped:", a, b)

x = y = z = 0
yap(x, y, z)
```

```
1 2
swapped: 2 1
0 0 0
```

Finding out what you have

`type()` tells you what kind of value something is, which is useful while you are learning:

```
yap(type(10))
yap(type(3.5))
yap(type("text"))
yap(type(nocap))
yap(type(npc))
yap(type(squad()))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
<class 'list'>
```

7. Numbers and arithmetic

The usual operators all work. Two are worth pointing out because beginners trip on them:

```
yap(7 + 2, 7 - 2, 7 * 2)
yap(7 / 2)
yap(7 // 2)
yap(7 % 2)
yap(7 ** 2)
```

```
9 5 14
3.5
3
1
49
```

- `/` always gives a decimal, even for numbers that divide evenly: `4 / 2` is `2.0`, not `2`.

- `//` throws away the remainder, and `%` keeps only the remainder. Together they answer 'how many times, and what is left over'.

`%%` is the workhorse of beginner programs, because *divisible by* means *remainder is zero*:

```
yap(10 % 2 == 0, 11 % 2 == 0)
yap("15 divisible by 3 and 5:", 15 % 3 == 0 and 15 % 5 == 0)
```

```
True False
15 divisible by 3 and 5: True
```

Handy number tools:

```
yap(abs(-4), round(3.7), round(3.14159, 2))
yap(min(4, 2, 9), max(4, 2, 9), sum([1, 2, 3]))
yap(int("42") + 1, float("2.5") * 2, str(99) + "!")
yap(divmod(17, 5))
```

```
4 4 3.14
2 9 6
43 5.0 99!
(3, 2)
```

Whole numbers in `W++` have no size limit:

```
yap(2 ** 128)
```

```
340282366920938463463374607431768211456
```

8. Text

Text goes in single or double quotes - use whichever avoids clashing with the quotes inside it:

```
a = "double quoted"
b = 'single quoted'
c = "she said \"hello\""
d = 'she said "hello"'
yap(a)
yap(b)
yap(c)
yap(d)
```

```
double quoted
single quoted
she said "hello"
she said "hello"
```

Triple quotes hold text that runs over several lines:

```
blurb = """line one
line two"""
yap(blurb)
```

```
line one
line two
```

Joining and repeating

```
first = "Ada"
last = "Lovelace"
yap(first + " " + last)
yap("ha" * 3)
yap("-" * 20)
```

```
Ada Lovelace
hahaha
-----
```

You cannot add text to a number with `+`. `"Score: " + 10` fails; write `"Score: " + str(10)`, or use **yap**'s several-values form, or an f-string.

f-strings: the tidy way to build text

Put `f` in front of a string and anything in curly braces is worked out and dropped in. This is the recommended way to build messages:

```

name = "Ada"
score = 91.5
items = squad([1, 2, 3])

yap(f"{name} scored {score}")
yap(f"{name} has {bodycount(items)} items")
yap(f"one more than the score is {score + 1}")
yap(f"rounded to one decimal: {score:.1f}")
yap(f"padded: [{name:>10}]")

```

```

Ada scored 91.5
Ada has 3 items
one more than the score is 92.5
rounded to one decimal: 91.5
padded: [      Ada]

```

The words inside the braces are real W++, so keywords work there too - notice **bodycount** above. Text outside the braces is left exactly as written, so a stray keyword in your sentence is safe:

```
yap(f"the word cook stays put, {1 + 1} does not")
```

```
the word cook stays put, 2 does not
```

Picking text apart

Text is a sequence, so you can index and slice it. Counting starts at zero, and negative numbers count from the end:

```

word = "playground"
yap(word[0], word[3], word[-1])
yap(word[0:4])
yap(word[4:])
yap(word[:-1])
yap(bodycount(word))

```

```

p y d
play
ground
dnuorgyalp
10

```

Useful text methods

```

messy = " Hello, World "
yap(f"[{messy.strip()}]")
yap(messy.strip().upper())
yap(messy.strip().lower())
yap("a,b,c".split(","))
yap("-".join(squad(["a", "b", "c"])))
yap("hello".replace("l", "L"))
yap("hello".startswith("he"), "hello".endswith("lo"))
yap("cook" in "cookbook", "steak" in "cookbook")
yap("hello".find("ll"), "Hello".capitalize())

```

```

[Hello, World]
HELLO, WORLD
hello, world
['a', 'b', 'c']
a-b-c
heLlo
True True
True False
2 Hello

```

9. Yes, no and nothing

Three keywords cover truth and emptiness:

W++	Means	Python
nocap	true	True
cap	false	False
npc	no value at all	None

```

ready = nocap
finished = cap
winner = npc

yap(ready, finished, winner)
yap(type(ready), type(winner))

```

```

True False None
<class 'bool'> <class 'NoneType'>

```

Comparing things

Comparisons produce **nocap** or **cap**:

```
yap(5 == 5, 5 != 5)
yap(5 > 3, 5 < 3, 5 >= 5, 5 <= 4)
yap("abc" == "abc", "abc" == "ABC")
```

```
True False
True False True False
True False
```

= assigns a value; == asks whether two things are equal. Mixing them up is the most common beginner slip in any language.

You can chain comparisons, which reads exactly how you would say it out loud:

```
age = 25
yap(18 <= age < 65)
```

```
True
```

Combining conditions

and, or and not stay as they are - they were never given nicknames:

```
age = 25
member = nocap

yap(age > 18 and member)
yap(age > 65 or member)
yap(not member)
```

```
True
True
False
```

Emptiness counts as false

Empty things behave as **cap** when used as a condition, which lets you write natural checks:

```

items = squad()
name = ""

bet not items:
    yap("the squad is empty")
bet not name:
    yap("no name given")

items.append("first")
bet items:
    yap("now it has", bodycount(items), "item")

```

```

the squad is empty
no name given
now it has 1 item

```

Zero, an empty string, an empty list, an empty dictionary and **npc** are all falsey. Everything else is truthy.

is and in

```

value = npc
yap(value is npc)
yap(value is not npc)

squad_goals = squad(["a", "b"])
yap("a" in squad_goals, "z" in squad_goals)
yap("z" not in squad_goals)

```

```

True
False
True False
True

```

Use `is npc` rather than `== npc` when checking for nothing. It is the habit Python programmers expect.

10. Making decisions

bet runs a block only when something is true. The block is the indented lines underneath it:

```
score = 91

bet score >= 50:
    yap("You passed")
    yap("Well done")
```

```
You passed
Well done
```

Indentation is not decoration in W++ - it is how the language knows which lines belong to the block. Use four spaces, and be consistent. Mixing tabs and spaces is an error.

nah covers everything else:

```
score = 32

bet score >= 50:
    yap("Pass")
nah:
    yap("Fail")
```

```
Fail
```

plotwist adds more questions in between. They are checked in order and the first true one wins:

```
cook grade(score):
    bet score >= 90:
        spill "A"
    plotwist score >= 80:
        spill "B"
    plotwist score >= 70:
        spill "C"
    plotwist score >= 50:
        spill "Pass"
    nah:
        spill "Fail"

spam mark in [95, 83, 71, 55, 20]:
    yap(mark, "->", grade(mark))
```

```
95 -> A
83 -> B
71 -> C
55 -> Pass
20 -> Fail
```

Because they are checked in order, you can write `score >= 80` for a B without also saying *and less than 90* - if it were 90 or more, the first question would already have answered.

Conditions inside conditions

```
age = 25
member = cap

bet age >= 18:
  bet member:
    yap("adult member")
  nah:
    yap("adult, not a member")
nah:
  yap("too young")
```

```
adult, not a member
```

Choosing a value in one line

When you only want to pick between two values, you can put the whole decision on one line. The shape is *value* **bet** *condition* **nah** *other value*:

```
score = 91
label = "pass" bet score >= 50 nah "fail"
yap(label)

count = 1
yap(f"{count} item" bet count == 1 nah f"{count} items")
```

```
pass
1 item
```

11. Repeating things

There are two loops. **spam** walks through a collection; **grind** keeps going while something stays true.

spam: once for each item

```
spam colour in squad(["red", "green", "blue"]):
  yap(colour)
```

```
red
green
blue
```

With **range** you loop a set number of times. `range(5)` counts 0, 1, 2, 3, 4 - it stops *before* the number you give it:


```

spam i in range(5):
    yap(i)

yap("---")
spam i in range(1, 6):
    yap(i)

yap("---")
spam i in range(0, 10, 3):
    yap(i)

yap("---")
spam i in range(3, 0, -1):
    yap(i)

```

```

0
1
2
3
4
---
1
2
3
4
5
---
0
3
6
9
---
3
2
1

```

You can walk through text, and through a dictionary:

```

spam letter in "wpp":
    yap(letter)

ages = tea(ada=36, alan=41)
spam name in ages:
    yap(name, "is", ages[name])

```

```

w
p
p
ada is 36
alan is 41

```

Knowing where you are: enumerate and zip

```
names = squad(["Ada", "Alan", "Grace"])

spam i, name in enumerate(names):
    yap(i, name)

yap("---")
spam i, name in enumerate(names, 1):
    yap(f"{i}. {name}")

yap("---")
scores = squad([91, 84, 99])
spam name, score in zip(names, scores):
    yap(name, score)
```

```
0 Ada
1 Alan
2 Grace
---
1. Ada
2. Alan
3. Grace
---
Ada 91
Alan 84
Grace 99
```

grind: keep going while true

```
countdown = 3
grind countdown > 0:
    yap(countdown)
    countdown = countdown - 1
yap("liftoff")
```

```
3
2
1
liftoff
```

Whatever the **grind** condition depends on must change inside the loop, or the loop never ends. Forgetting the `countdown = countdown - 1` line above is the classic way to write a program that runs forever. In the playground a runaway loop is stopped for you after ten seconds; in the terminal press Ctrl+C.

dip and skrrt

dip leaves the loop immediately. **skrrt** skips the rest of this round and starts the next one:

```

spam n in range(1, 11):
    bet n % 2 == 0:
        skrrt
    bet n > 7:
        dip
    yap(n)

```

```

1
3
5
7

```

A common pattern is **grind nocap** - loop forever, and **dip** out when you are done. This is how you keep asking until the answer makes sense:

```

grind nocap:
    answer = dm("Type yes to continue: ")
    bet answer == "yes":
        dip
    yap("That was not yes. Try again.")
yap("Continuing")

```

```

Type yes to continue: no
That was not yes. Try again.
Type yes to continue: maybe
That was not yes. Try again.
Type yes to continue: yes
Continuing

```

A loop can have a nah too

Attached to a loop, **nah** runs when the loop finished normally - meaning it was never cut short by **dip**. It is perfect for searching:

```

numbers = squad([4, 9, 16, 25])

spam n in numbers:
    bet n % 2 == 1:
        yap("found an odd one:", n)
    dip
nah:
    yap("they were all even")

```

```

found an odd one: 9

```

Loops inside loops

```

spam row in range(1, 4):
    line = ""
    spam col in range(1, 4):
        line = line + f"{row * col:3}"
    yap(line)

```

```

1 2 3
2 4 6
3 6 9

```

12. Collections

Three keywords cover the three collections you will use constantly. **bodycount** tells you how many items any of them holds.

W++	What it is	Good for	Written as
squad	a list	an ordered run of things	[1, 2, 3]
tea	a dictionary	looking things up by name	{"a": 1}
cult	a set	unique things, fast checks	{1, 2, 3}

squad: lists

A list keeps things in order. You can build one with square brackets or with **squad**:

```

empty = squad()
numbers = squad([3, 1, 2])
letters = ["a", "b", "c"]
mixed = [1, "two", nocap, npc]

yap(empty, numbers, letters, mixed)
yap(bodycount(numbers))
yap(squad(range(5)))
yap(squad("abc"))

```

```

[] [3, 1, 2] ['a', 'b', 'c'] [1, 'two', True, None]
3
[0, 1, 2, 3, 4]
['a', 'b', 'c']

```

Reach into a list by position, starting at zero:

```

days = squad(["mon", "tue", "wed", "thu", "fri"])

yap(days[0], days[2], days[-1])
yap(days[1:3])
yap(days[:2])
yap(days[-2:])

days[0] = "MON"
yap(days)

```

```

mon wed fri
['tue', 'wed']
['mon', 'tue']
['thu', 'fri']
['MON', 'tue', 'wed', 'thu', 'fri']

```

Changing a list:

```

items = squad(["b"])

items.append("c")
items.insert(0, "a")
yap(items)

items.extend(["d", "e"])
yap(items)

removed = items.pop()
yap("removed", removed, "leaving", items)

items.remove("a")
yap(items)

yap("index of c:", items.index("c"))
yap("how many b:", items.count("b"))

```

```

['a', 'b', 'c']
['a', 'b', 'c', 'd', 'e']
removed e leaving ['a', 'b', 'c', 'd']
['b', 'c', 'd']
index of c: 1
how many b: 1

```

Sorting and reversing:

```

numbers = squad([3, 1, 4, 1, 5])

yap(sorted(numbers))
yap(sorted(numbers, reverse=nocap))
yap("original untouched:", numbers)

numbers.sort()
yap("now sorted in place:", numbers)

words = squad(["pear", "fig", "banana"])
yap(sorted(words, key=bodycount))
yap(squad(reversed(words)))
yap(sum(numbers), min(numbers), max(numbers))

```

```

[1, 1, 3, 4, 5]
[5, 4, 3, 1, 1]
original untouched: [3, 1, 4, 1, 5]
now sorted in place: [1, 1, 3, 4, 5]
['fig', 'pear', 'banana']
['banana', 'fig', 'pear']
14 1 5

```

tea: dictionaries

A dictionary stores values under names, so you can look them up instantly. Each entry is a *key* and a *value*:

```

scores = tea(ada=91, alan=84)
also = {"grace": 99, "edsgen": 77}

yap(scores)
yap(also)
yap(scores["ada"])
yap(bodycount(scores))

```

```

{'ada': 91, 'alan': 84}
{'grace': 99, 'edsgen': 77}
91
2

```

Adding, changing and removing entries:

```

player = tea()

player["name"] = "Ada"
player["score"] = 0
yap(player)

player["score"] = player["score"] + 10
yap(player)

del player["score"]
yap(player)

```

```

{'name': 'Ada', 'score': 0}
{'name': 'Ada', 'score': 10}
{'name': 'Ada'}

```

Asking for a key that is not there is an error. **get** is the safe way, and it lets you supply a fallback:

```

scores = tea(ada=91)

yap(scores.get("ada"))
yap(scores.get("nobody"))
yap(scores.get("nobody", 0))
yap("ada" in scores, "nobody" in scores)

```

```

91
None
0
True False

```

Looping over a dictionary:

```

scores = tea(ada=91, alan=84, grace=99)

spam name in scores:
    yap(name)

yap("---")
spam name, score in scores.items():
    yap(name, score)

yap("---")
yap(sorted(scores.keys()))
yap(sorted(scores.values()))
yap("best:", max(scores, key=scores.get))

```

```

ada
alan
grace
---
ada 91
alan 84
grace 99
---
['ada', 'alan', 'grace']
[84, 91, 99]
best: grace

```

cult: sets

A set holds each item only once and answers *is this in there?* very quickly. It has no order:

```

tags = cult(["red", "blue", "red", "green"])
yap(bodycount(tags))
yap(sorted(tags))

tags.add("blue")
yap("adding a duplicate changes nothing:", bodycount(tags))

tags.add("pink")
tags.discard("red")
yap(sorted(tags))
yap("blue" in tags)

```

```

3
['blue', 'green', 'red']
adding a duplicate changes nothing: 3
['blue', 'green', 'pink']
True

```

The classic use is removing duplicates from a list:


```

numbers = squad([3, 1, 3, 2, 1, 3])
unique = squad(cult(numbers))
yap(sorted(unique))

```

```
[1, 2, 3]
```

Sets can be compared against each other:

```

a = cult([1, 2, 3, 4])
b = cult([3, 4, 5])

yap("in both:", sorted(a & b))
yap("in either:", sorted(a | b))
yap("only in a:", sorted(a - b))
yap("in one but not both:", sorted(a ^ b))

```

```

in both: [3, 4]
in either: [1, 2, 3, 4, 5]
only in a: [1, 2]
in one but not both: [1, 2, 5]

```

Tuples: fixed little groups

A tuple is like a list that cannot be changed. It has no W++ nickname; write it with round brackets. You have already seen them coming out of `enumerate` and `items`:

```

point = (3, 4)
yap(point, point[0], bodycount(point))

x, y = point
yap("unpacked:", x, y)

cook middle(a, b):
    spill (a + b) / 2, a * b

average, product = middle(4, 6)
yap(average, product)

```

```

(3, 4) 3 2
unpacked: 3 4
5.0 24

```

Collections inside collections

```

grid = squad([[1, 2, 3], [4, 5, 6]])
yap(grid[0])
yap(grid[1][2])

people = squad([
    tea(name="Ada", langs=squad(["W++", "Python"])),
    tea(name="Alan", langs=squad(["Maths"])),
])

spam person in people:
    yap(person["name"], "knows", ", ".join(person["langs"]))

```

```

[1, 2, 3]
6
Ada knows W++, Python
Alan knows Maths

```

13. Building collections in one line

A *comprehension* builds a collection from another one. It is the same **spam** you already know, written inside the brackets. Start with the long way:

```

squares = squad()
spam n in range(6):
    squares.append(n * n)
yap(squares)

```

```
[0, 1, 4, 9, 16, 25]
```

Now the short way. Read it as *n times n, for each n in range(6)*:

```
yap([n * n spam n in range(6)])
```

```
[0, 1, 4, 9, 16, 25]
```

Add a **bet** at the end to keep only some of the items:

```

yap([n spam n in range(20) bet n % 3 == 0])
words = squad(["fig", "banana", "pear", "kiwi"])
yap([w.upper() spam w in words bet bodycount(w) > 3])

```

```

[0, 3, 6, 9, 12, 15, 18]
['BANANA', 'PEAR', 'KIWI']

```

Put a one-line decision in the front half to transform items differently:

```
yap(["even" bet n % 2 == 0 nah "odd" spam n in range(5)])
```

```
['even', 'odd', 'even', 'odd', 'even']
```

The same shape builds dictionaries and sets:

```
words = squad(["fig", "banana", "pear"])  
  
yap({w: bodycount(w) spam w in words})  
yap(sorted({bodycount(w) spam w in words}))
```

```
{'fig': 3, 'banana': 6, 'pear': 4}  
[3, 4, 6]
```

Without brackets around it, the same expression can be fed straight into something that adds things up - no list is built at all, which matters when the numbers get large:

```
yap(sum(n spam n in range(1, 101)))  
yap(max(bodycount(w) spam w in squad(["fig", "banana"])))  
yap(any(n > 90 spam n in squad([10, 95])))  
yap(all(n > 90 spam n in squad([10, 95])))
```

```
5050  
6  
True  
False
```

Comprehensions can nest, which is how you flatten or build grids:

```
grid = squad([[1, 2], [3, 4]])  
yap([cell spam row in grid spam cell in row])  
yap([[r * c spam c in range(1, 4)] spam r in range(1, 4)])
```

```
[1, 2, 3, 4]  
[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

Comprehensions are lovely for one clear step. If you need two steps and a couple of conditions, a normal **spam** loop is easier to read - and easier to fix later.

14. Functions

A function is a named piece of program you can run whenever you like. **cook** starts one and **spill** sends a value back:

```
cook greet(name):  
    spill "Hello, " + name  
  
yap(greet("Ada"))  
yap(greet("Alan"))
```

```
Hello, Ada  
Hello, Alan
```

The names in the brackets are *parameters* - stand-ins filled with whatever you pass in. A function without a **spill** simply does its work and hands back **npc**:

```
cook announce(text):  
    yap("***", text, "***")  
  
result = announce("no spill here")  
yap("it gave back:", result)
```

```
*** no spill here ***  
it gave back: None
```

Several parameters, and defaults

```
cook power(base, exponent=2):  
    spill base ** exponent  
  
yap(power(5))  
yap(power(5, 3))  
yap(power(exponent=3, base=2))
```

```
25  
125  
8
```

Naming your arguments as in `power(base=2)` makes a call much easier to read when a function takes several things.

spill can hand back several values at once:

```

cook stats(numbers):
    spill min(numbers), max(numbers), sum(numbers) / bodycount(numbers)

low, high, mean = stats(squad([4, 8, 15, 16]))
yap(low, high, mean)

```

```

4 16 10.75

```

spill also ends the function on the spot, which is handy for dealing with awkward cases first:

```

cook safe_divide(a, b):
    bet b == 0:
        spill npc
    spill a / b

yap(safe_divide(10, 2))
yap(safe_divide(10, 0))

```

```

5.0
None

```

Taking any number of arguments

```

cook total(*numbers):
    spill sum(numbers)

yap(total(1, 2), total(1, 2, 3, 4))

cook describe(**details):
    spam key, value in details.items():
        yap(key, "=", value)

describe(name="Ada", score=91)

```

```

3 10
name = Ada
score = 91

```

Functions calling themselves

A function may call itself. Always give it a case that stops:

```

cook factorial(n):
    bet n <= 1:
        spill 1
    spill n * factorial(n - 1)

yap(factorial(5))
yap([factorial(n) spam n in range(1, 8)])

```

```

120
[1, 2, 6, 24, 120, 720, 5040]

```

Tiny throwaway functions

lambda makes a one-expression function without a name. It is mostly used to tell sorted what to sort by:

```

double = lambda n: n * 2
yap(double(21))

people = squad([
    tea(name="Ada", score=91),
    tea(name="Alan", score=84),
    tea(name="Grace", score=99),
])

ranked = sorted(people, key=lambda person: person["score"], reverse=nocap)
spam person in ranked:
    yap(person["name"], person["score"])

```

```

42
Grace 99
Ada 91
Alan 84

```

Explaining yourself

A string on the first line of a function is a note for whoever reads it next - often you:

```

cook celsius_to_f(c):
    "Convert a Celsius temperature to Fahrenheit."
    spill c * 9 / 5 + 32

yap(celsius_to_f(100))
yap(celsius_to_f.__doc__)

```

```

212.0
Convert a Celsius temperature to Fahrenheit.

```

15b. Where names live

A name made inside a function belongs to that function and disappears when it ends:

```
cook demo():
    inside = "local"
    yap("in the function:", inside)

demo()
yap("outside, asking for it:")
```

```
in the function: local
outside, asking for it:
```

Reading an outer name is fine; *changing* one needs permission:

```
total = 0

cook add_wrong(n):
    total = n

cook add_right(n):
    global total
    total = total + n

add_wrong(5)
yap("after add_wrong:", total)

add_right(5)
add_right(3)
yap("after add_right:", total)
```

```
after add_wrong: 0
after add_right: 8
```

A function inside a function can remember the outer one's values. `nonlocal` lets it change them:

```
cook make_counter():
    count = 0
    cook step():
        nonlocal count
        count = count + 1
        spill count
    spill step

tick = make_counter()
yap(tick(), tick(), tick())
```

```
1 2 3
```

Reach for global sparingly. Passing values in and spilling results back is easier to follow.

15. When things go wrong

W++ replaces Python's tracebacks with its own messages, called the **Skill Issue Protocol**. Here is one:

```
yap("this line is fine")
yap(undefined_name)
```

```
this line is fine
Bro is making up words now (NameError)
  where: example.wpp, line 2
        2 | yap(undefined_name)
  details: name 'undefined_name' is not defined
```

Read it from the top: *what* went wrong, *where*, the line itself, then Python's own description. The line number is the line in your file, so you can go straight to it.

In a real terminal each message is preceded by a siren emoji. It is left out of this guide only because the PDF fonts cannot draw it.

The seven messages

When Python says	W++ says
SyntaxError	Negative Aura: Bro forgot how to type (SyntaxError)
NameError	Bro is making up words now (NameError)
TypeError	Oil up bro, you can't combine those (TypeError)
IndexError	Blud thinks he has more items than he does (IndexError)
ZeroDivisionError	Bro just broke the matrix (ZeroDivisionError)
IndentationError	Your spaces are looking a little sus (IndentationError)
KeyboardInterrupt	Go touch grass, you've been looping forever (KeyboardInterrupt)

Anything outside this table is reported as *Unspecified skill issue* with the real name in brackets, so you are never left without a clue:


```
t = tea(a=1)
yap(t["missing"])
```

```
Unspecified skill issue (KeyError)
  where: example.wpp, line 2
        2 | yap(t["missing"])
  details: 'missing'
```

The mistakes you will actually make

A name that does not exist. Usually a typo, or using something before you set it:

```
total = 10
yap(totl)
```

```
Bro is making up words now (NameError)
  where: example.wpp, line 2
        2 | yap(totl)
  details: name 'totl' is not defined
```

Mixing text and numbers. Convert one of them:

```
yap("Score: " + 10)
```

```
Oil up bro, you can't combine those (TypeError)
  where: example.wpp, line 1
        1 | yap("Score: " + 10)
  details: can only concatenate str (not "int") to str
```

Reaching past the end of a list. Five items means positions 0 to 4:

```
items = squad(["a", "b"])
yap(items[5])
```

```
Blud thinks he has more items than he does (IndexError)
  where: example.wpp, line 2
        2 | yap(items[5])
  details: list index out of range
```

Dividing by zero. Check the divisor first:

```
yap(10 / 0)
```

```
Bro just broke the matrix (ZeroDivisionError)
  where: example.wpp, line 1
        1 | yap(10 / 0)
  details: division by zero
```

Indentation that does not line up. Keep to four spaces per level and never mix in tabs:

```
cook f():
  yap(1)
  yap(2)
```

```
Your spaces are looking a little sus (IndentationError)
  where: example.wpp, line 3
        3 | yap(2)
  details: unexpected indent
```

Handling errors instead of crashing

Wrap the risky part in try and say what to do when it fails. This is how you deal with a user typing nonsense:

```
cook read_number(text):
  try:
    spill int(text)
  except ValueError:
    yap("that was not a number:", text)
    spill npc

yap(read_number("42"))
yap(read_number("banana"))
```

```
42
that was not a number: banana
None
```

A fuller shape: **nah** runs when nothing went wrong, and **finally** runs either way:

```

cook divide(a, b):
    try:
        answer = a / b
    except ZeroDivisionError:
        yap("cannot divide by zero")
    nah:
        yap("got", answer)
    finally:
        yap("done with", a, b)

divide(10, 2)
divide(10, 0)

```

```

got 5.0
done with 10 2
cannot divide by zero
done with 10 0

```

Catch several kinds at once, and look at the problem itself:

```

cook attempt(items, index):
    try:
        spill items[index]
    except (IndexError, TypeError) as problem:
        spill f"failed: {problem}"

data = squad(["a", "b"])
yap(attempt(data, 0))
yap(attempt(data, 9))
yap(attempt(data, "x"))

```

```

a
failed: list index out of range
failed: list indices must be integers or slices, not str

```

You can raise problems yourself when your function is given something it cannot work with:

```

cook set_age(age):
    bet age < 0:
        raise ValueError("age cannot be negative")
    spill age

yap(set_age(30))
try:
    set_age(-1)
except ValueError as problem:
    yap("refused:", problem)

```

```

30
refused: age cannot be negative

```

The loop that keeps asking until the input is valid is worth memorising - it combines almost everything so far:

```
grind nocap:
  raw = dm("Age: ")
  try:
    age = int(raw)
  except ValueError:
    yap("Please type digits.")
    skrrt
  bet age < 0:
    yap("Please type a positive number.")
    skrrt
  dip

yap("Thank you, age is", age)
```

```
Age: abc
Please type digits.
Age: -3
Please type a positive number.
Age: 30
Thank you, age is 30
```

16. Reserved words: the one real gotcha

This is the only place W++ can surprise you, so it gets its own section. The nineteen keywords are **reserved**: you cannot use one as the name of your own variable, function, class or parameter.

The reason is simple once you have seen the pipeline in section 1. If you write a function called **dip**, the translator turns every **dip** into `break` - both where you defined it and everywhere you called it. There is no way to tell the two apart. Python has the same rule about its own words: you cannot write `def break()` either.

W++ tells you plainly when you try:

```
class Stack:
  cook dip(self):
    spill "pop"
```

```
Negative Aura: Bro forgot how to type (SyntaxError)
where: example.wpp, line 2
2 | cook dip(self):
details: 'dip' is a W++ keyword (it becomes Python's 'break'), so it cannot be
used as a name
```

What is fine

Only the exact words are taken. Anything longer is yours:

```

cookie = "fine"
capacity = 100
recap = "also fine"
bet_size = 5
my_squad = squad([1, 2])
spam_folder = squad()

yap(cookie, capacity, recap, bet_size, my_squad, spam_folder)

```

```
fine 100 also fine 5 [1, 2] []
```

Names after a dot are never keywords, so object data and methods can be called anything:

```

class Bottle:
    cook __init__(self):
        self.cap = "on"
        self.range = squad([1, 2])

b = Bottle()
yap(b.cap, b.range)

```

```
on [1, 2]
```

If you want a name that is taken

Add a word, or use a synonym. All of these are ordinary names:

wanted	use instead
dip	pop, remove, leave, exit_loop
cap	limit, maximum, lid
spill	give, emit, output
range	span, bounds, extent
squad	group, team, my_squad
tea	table, lookup, mapping
cult	unique, distinct

The full reserved list

W++ word	Becomes
cook	def
spill	return
yap	print
dm	input
bodycount	len
bet	if
plotwist	elif
nah	else
spam	for
grind	while
dip	break
skrrt	continue
nocap	True
cap	False
npc	None
squad	list
tea	dict
cult	set
range	range

Python's own keywords are also reserved, exactly as in Python: `and`, `or`, `not`, `in`, `is`, `if`, `for`, `while`, `def`, `class`, `return`, `import`, `try`, `except`, `finally`, `with`, `as`, `lambda`, `global`, `nonlocal`, `yield`, `raise`, `assert`, `pass`, `del`, `None`, `True`, `False`. You write most of these as their W++ nicknames anyway.

17. Everything Python brings with it

Nineteen words got nicknames. Everything else in Python is untouched and available. This is what makes W++ able to build real things, and it is worth skimming so you know what you already have.

Words that stay exactly as they are

```
and    or    not    in    is    None-checks via npc
class  import from  as    pass  del
try    except finally raise  assert with
lambda yield global nonlocal
match case  async  await
```

Built-in functions you will use

Function	What it does
int, float, str, bool	convert between kinds of value
sum, min, max	add up, smallest, largest
sorted, reversed	ordered copy, reversed copy
enumerate, zip	count while looping, pair up two lists
abs, round, divmod	size, rounding, quotient and remainder
any, all	is one true, are they all true
map, filter	apply to each, keep some
type, isinstance	what kind of value is this
input, print, len	the plain names behind dm, yap, bodycount

The plain Python names still work. `len(x)` and `bodycount(x)` do the same thing, and `list` and **squad** are the same function. Use the W++ words - they are the point - but nothing breaks if a plain one slips in.

The standard library

`import` gives you Python's whole library. Nothing to install:

```
import math
import random

yap(math.sqrt(144), math.pi)
yap(math.floor(3.7), math.ceil(3.2))

random.seed(7)
yap(random.randint(1, 6))
yap(random.choice(squad(["rock", "paper", "scissors"])))
```

```
12.0 3.141592653589793
3 4
3
rock
```

Other modules worth knowing early:

```
from collections import Counter
import json

words = "the cook will cook the meal".split()
yap(Counter(words).most_common(2))

data = tea(name="Ada", scores=squad([1, 2]))
text = json.dumps(data)
yap(text)
yap(json.loads(text)["name"])
```

```
[('the', 2), ('cook', 2)]
{"name": "Ada", "scores": [1, 2]}
Ada
```

```
import datetime

day = datetime.date(2026, 1, 31)
yap(day.year, day.month, day.day)
yap(day.strftime("%d %B %Y"))
yap(day + datetime.timedelta(days=1))
```

```
2026 1 31
31 January 2026
2026-02-01
```

A short list to explore when you need it:

Module	For
math	square roots, trigonometry, constants
random	dice, shuffling, random choices

Module	For
datetime	dates, times, differences
json	saving and loading structured data
collections	Counter, defaultdict, deque
itertools	combinations, permutations, cycles
re	finding patterns in text
os, pathlib	files and folders
statistics	mean, median, standard deviation
time	timing things, pausing

Reading and writing files

Files work exactly as in Python. `with` closes the file for you:

```
with open("notes.txt", "w") as handle:
    handle.write("first line\n")
    handle.write("second line\n")

with open("notes.txt") as handle:
    for line in handle:
        yap(line.strip())
```

16b. Your own kinds of thing

A `class` describes a kind of thing: the data each one carries and what it can do. `class` itself has no nickname, but the methods inside it are made with **`cook`**:

```

class Dog:
    cook __init__(self, name):
        self.name = name
        self.tricks = squad()

    cook learn(self, trick):
        self.tricks.append(trick)

    cook show_off(self):
        bet not self.tricks:
            spill f"{self.name} knows nothing yet"
        spill f"{self.name} can " + ", ".join(self.tricks)

rex = Dog("Rex")
rex.learn("sit")
rex.learn("roll over")
yap(rex.show_off())

fido = Dog("Fido")
yap(fido.show_off())

```

```

Rex can sit, roll over
Fido knows nothing yet

```

- `__init__` runs when you make a new one, and sets up its data.
- `self` is the particular object being worked on. Every method takes it as the first parameter, and you never pass it yourself.
- `self.name` is data belonging to that one object - `rex` and `fido` have their own.

Printing your objects nicely

Give a class `__str__` and it decides what **yap** shows:

```

class Point:
    cook __init__(self, x, y):
        self.x = x
        self.y = y

    cook __str__(self):
        spill f"({self.x}, {self.y})"

    cook __eq__(self, other):
        spill self.x == other.x and self.y == other.y

a = Point(1, 2)
b = Point(1, 2)
yap(a)
yap(a == b, a is b)

```

```

(1, 2)
True False

```

Building on another class

```

class Animal:
    cook __init__(self, name):
        self.name = name

    cook speak(self):
        spill "..."

    cook introduce(self):
        spill f"{self.name} says {self.speak()}"

class Cat(Animal):
    cook speak(self):
        spill "meow"

class LoudCat(Cat):
    cook speak(self):
        spill super().speak().upper() + "!"

spam pet in squad([Animal("Thing"), Cat("Tom"), LoudCat("Rex")]):
    yap(pet.introduce())

```

```

Thing says ...
Tom says meow
Rex says MEOW!

```

A method cannot be named after a W++ keyword - cook `dip(self)` is refused, because **dip** is reserved. Data attributes are fine: `self.cap = 10` works, because a name after a dot is never a keyword. Section 16 has the details.

17b. Producing values lazily

A function that uses `yield` instead of **spill** hands back values one at a time, only as they are asked for. It is how you work with something long - or endless - without building it all first:

```
cook countdown(n):  
  grind n > 0:  
    yield n  
    n = n - 1  
  
spam value in countdown(4):  
  yap(value)  
  
yap(squad(countdown(3)))
```

```
4  
3  
2  
1  
[3, 2, 1]
```

Because nothing is computed until it is needed, an endless generator is perfectly safe as long as you stop taking from it:

```
cook evens():  
  n = 0  
  grind nocap:  
    yield n  
    n = n + 2  
  
picked = squad()  
spam value in evens():  
  bet bodycount(picked) == 5:  
    dip  
    picked.append(value)  
yap(picked)
```

```
[0, 2, 4, 6, 8]
```

`yield` from passes along everything another generator produces:

```
cook first_part():
    yield "a"
    yield "b"

cook everything():
    yield from first_part()
    yield "c"

yap(squad(everything()))
```

```
['a', 'b', 'c']
```

18. Tools that help

Seeing the Python behind your program

This is the single most useful thing you can do when a program confuses you. `--emit` prints the Python that your W++ turned into, without running it:

```
$ python wpp.py --emit examples/vibe_check.wpp
```

```
def check_vibe(name):
    if name == "Claude":
        return "W AI"
    else:
        return "Mid"

username = input("Who are you? ")
print("Vibe check: ", check_vibe(username))
```

If the Python looks right, the bug is in your logic. If the Python looks wrong, look for a keyword you used as a name.

The dictionary, on demand

```
$ python wpp.py --keywords
```

Prints the whole table from section 3 in your terminal.

Everything the command offers

Command	What it does
<code>python wpp.py FILE.wpp</code>	run a program
<code>python wpp.py --emit FILE.wpp</code>	show the generated Python
<code>python wpp.py --keywords</code>	print the keyword table
<code>python wpp.py --version</code>	print the W++ version
<code>python playground/server.py</code>	start the web playground

Exit codes

Useful if you ever run W++ from a script:

Code	Meaning
0	the program finished normally
1	a skill issue stopped it
2	bad command line, or the file was not found
130	interrupted with Ctrl+C

In the playground

- **Run** or Ctrl+Enter runs what is in the editor.
- **Examples** loads a ready-made program.
- **Documentation** shows the keyword and error tables.
- **Stop** ends a program that is still going.
- **Copy** and **Reset** act on the editor.
- When a program calls **dm**, type your answer directly in the output panel and press Enter.
- A runaway loop is stopped after ten seconds of running - time spent waiting for you to type does not count.

19. Complete programs

Everything up to here, put to work. Each program is short enough to read in one go and does something real.

Number guessing game

Loops, input, decisions and a running count:

```

import random

random.seed(3)
secret = random.randint(1, 50)
tries = 0

yap("I picked a number between 1 and 50.")

grind nocap:
    guess = int(dm("Your guess: "))
    tries = tries + 1

    bet guess < secret:
        yap("Higher.")
    plotwist guess > secret:
        yap("Lower.")
    nah:
        yap(f"Got it in {tries} tries. It was {secret}.")
        dip

```

```

I picked a number between 1 and 50.
Your guess: 25
Lower.
Your guess: 12
Higher.
Your guess: 18
Lower.
Your guess: 15
Higher.
Your guess: 16
Got it in 5 tries. It was 16.

```

Grade book

A dictionary of lists, and a report built from it:

```

grades = tea()
grades["Ada"] = squad([91, 88, 95])
grades["Alan"] = squad([72, 80, 68])
grades["Grace"] = squad([100, 98, 99])

cook average(marks):
    spill sum(marks) / bodycount(marks)

cook letter(mark):
    bet mark >= 90:
        spill "A"
    plotwist mark >= 80:
        spill "B"
    plotwist mark >= 70:
        spill "C"
    nah:
        spill "F"

yap(f"{'Name':<8}{ 'Avg':>7} Grade")
yap("-" * 24)

ranked = sorted(grades.items(), key=lambda entry: average(entry[1]), reverse=nocap)
spam name, marks in ranked:
    mean = average(marks)
    yap(f"{'name':<8}{mean:>7.1f} {letter(mean)}")

everyone = [mark spam marks in grades.values() spam mark in marks]
yap("-" * 24)
yap(f"class average: {average(everyone):.1f}")
yap(f"best single mark: {max(everyone)}")

```

```

Name      Avg  Grade
-----
Grace     99.0  A
Ada       91.3  A
Alan      73.3  C
-----
class average: 87.9
best single mark: 100

```

Word frequency counter

Text in, ranked counts out:


```

text = """
the quick brown fox jumps over the lazy dog
the dog barks and the fox runs
"""

counts = tea()
spam word in text.lower().split():
    clean = word.strip(".,!?")
    bet not clean:
        skrrrt
    counts[clean] = counts.get(clean, 0) + 1

ranked = sorted(counts.items(), key=lambda pair: (-pair[1], pair[0]))

yap(f"{bodycount(counts)} different words")
spam word, n in ranked[:5]:
    bar = "#" * n
    yap(f"{word:<8}{n:>3} {bar}")

```

```

11 different words
the      4 ####
dog      2 ##
fox      2 ##
and      1 #
barks    1 #

```

To-do list

A list of dictionaries, and functions that work on it:

```

tasks = squad()

cook add(title, priority="normal"):
    tasks.append(tea(title=title, priority=priority, done=cap))

cook finish(title):
    spam task in tasks:
        bet task["title"] == title:
            task["done"] = nocap
            spill nocap
    spill cap

cook show():
    bet not tasks:
        yap("nothing to do")
        spill
    order = tea(high=0, normal=1, low=2)
    spam task in sorted(tasks, key=lambda t: order[t["priority"]]):
        mark = "x" bet task["done"] nah " "
        yap(f"[{mark}] {task['title']} ({task['priority']})")

add("write the guide", "high")
add("water the plants", "low")
add("review pull request")

finish("water the plants")
yap("found the task:", finish("not a real task"))

show()
left = bodycount([t spam t in tasks bet not t["done"]])
yap(f"{left} still open")

```

```

found the task: False
[ ] write the guide (high)
[ ] review pull request (normal)
[x] water the plants (low)
2 still open

```

Bank account, as a class

```

class Account:
    cook __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        self.history = squad()

    cook deposit(self, amount):
        bet amount <= 0:
            raise ValueError("deposit must be positive")
        self.balance = self.balance + amount
        self.history.append(f"+{amount}")

    cook withdraw(self, amount):
        bet amount > self.balance:
            spill cap
        self.balance = self.balance - amount
        self.history.append(f"-{amount}")
        spill nocap

    cook __str__(self):
        spill f"{self.owner}: {self.balance}"

account = Account("Ada", 100)
account.deposit(50)

yap(account)
yap("withdraw 30:", account.withdraw(30))
yap("withdraw 500:", account.withdraw(500))
yap(account)
yap("history:", ", ".join(account.history))

try:
    account.deposit(-5)
except ValueError as problem:
    yap("refused:", problem)

```

```

Ada: 150
withdraw 30: True
withdraw 500: False
Ada: 120
history: +50, -30
refused: deposit must be positive

```

FizzBuzz, three ways

The classic, written normally, as a function, and squeezed into one line:

```

spam i in range(1, 16):
    bet i % 15 == 0:
        yap("FizzBuzz")
    plotwist i % 3 == 0:
        yap("Fizz")
    plotwist i % 5 == 0:
        yap("Buzz")
    nah:
        yap(i)

```

```

1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz

```

```

cook fizzbuzz(n):
    bet n % 15 == 0:
        spill "FizzBuzz"
    plotwist n % 3 == 0:
        spill "Fizz"
    plotwist n % 5 == 0:
        spill "Buzz"
    spill str(n)

yap(" ".join([fizzbuzz(n) spam n in range(1, 16)]))

```

```

1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz

```

Sorting, from scratch

Recursion and comprehensions together:

```

cook quicksort(items):
  bet bodycount(items) <= 1:
    spill items
  pivot = items[bodycount(items) // 2]
  smaller = [i spam i in items bet i < pivot]
  same = [i spam i in items bet i == pivot]
  bigger = [i spam i in items bet i > pivot]
  spill quicksort(smaller) + same + quicksort(bigger)

yap(quicksort(squad([5, 3, 8, 1, 9, 2, 7, 3])))

```

```
[1, 2, 3, 3, 5, 7, 8, 9]
```

Prime numbers

```

cook primes_upto(limit):
  maybe = [nocap] * (limit + 1)
  maybe[0] = cap
  bet limit >= 1:
    maybe[1] = cap

  n = 2
  grind n * n <= limit:
    bet maybe[n]:
      multiple = n * n
      grind multiple <= limit:
        maybe[multiple] = cap
        multiple = multiple + n
    n = n + 1

  spill [i spam i in range(limit + 1) bet maybe[i]]

found = primes_upto(60)
yap(f"{bodycount(found)} primes up to 60")
yap(found)

```

```

17 primes up to 60
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59]

```

Finding a way through a graph

Sets, lists as queues, and a search that ends when it should:

```

routes = tea()
routes["home"] = squad(["shop", "park"])
routes["shop"] = squad(["school"])
routes["park"] = squad(["school", "lake"])
routes["school"] = squad(["library"])
routes["lake"] = squad([])
routes["library"] = squad([])

cook shortest(start, goal):
    queue = squad([squad([start])])
    seen = cult([start])

    grind bodycount(queue) > 0:
        path = queue.pop(0)
        here = path[-1]

        bet here == goal:
            spill path

        spam nxt in routes.get(here, squad()):
            bet nxt in seen:
                skrrt
            seen.add(nxt)
            queue.append(path + squad([nxt]))

    spill npc

yap(" -> ".join(shortest("home", "library")))
yap("to the moon:", shortest("home", "moon"))

```

```

home -> shop -> school -> library
to the moon: None

```

20. Habits worth picking up

- **Four spaces per level of indentation.** Never mix tabs and spaces in one file.
- **Name things for what they hold.** `player_score` beats `ps`, and you will thank yourself in a week.
- **Use lower_case_with_underscores** for variables and functions, and **CapitalWords** for classes. That is what Python programmers expect.
- **Leave comments for why, not what.** The code already says what it does.
- **Keep functions small.** If one does not fit on a screen, it is probably two functions.
- **Print things while you work.** A quick **yap** showing a value is the fastest way to find out where a program went wrong.
- **Reach for --emit when stuck.** Seeing the Python usually makes the problem obvious.
- **Read the error, all of it.** The message says what, the line says where, and the details say why.

Comments

```
# A full line comment.  
yap("code") # or at the end of a line  
  
# Keywords inside comments are ignored:  
# cook spill yap bet nah  
yap("still fine")  
  
code  
still fine
```

A note on speed

W++ runs at Python's speed, because it *is* Python by the time it executes. The keyword swap happens once, before your program starts, and costs nothing measurable. Nothing in this guide is slower than the Python it becomes.

21. What W++ does not do

An honest list, so nothing catches you out:

- **Keywords are reserved.** Section 16 covers this. It is the one thing you will hit.
- **Only the nineteen words have nicknames.** `and`, `or`, `not`, `class`, `import`, `try` and the rest are written the Python way.
- **One file per program.** You can import Python modules, but you cannot yet import another `.wpp` file.
- **No interactive prompt.** There is no W++ REPL; use a file or the playground.
- **One error at a time.** The first failure stops the program, just as in Python.
- **Keywords are syntax, not a display layer.** `yap(nocap)` prints `True`.
- **The playground is a local tool.** It runs code as you, with only a time limit, so keep it on your own machine.

None of these limit what you can build. The standard library, classes, generators and the rest of Python are all available - so the ceiling on what you can make in W++ is your imagination, not the language.

22. Where to go next

You now have the whole language. The fastest way to make it stick is not to read more - it is to break things on purpose.

- Open the playground, load an example, and change it until it stops working. Read the error, fix it, change it again. That loop is how programmers actually learn.

- Pick one program from section 19 and add a feature to it. Give the grade book a pass mark you can set. Let the to-do list save to a file with json.
- When something confuses you, run `--emit` and read the Python. It is the quickest way to see what your code really said.
- Write something you actually want. A dice roller, a quiz, a budget tracker, a text adventure. Everything in this guide was chosen because it turns up in real little programs.

And remember the one rule that unlocks the rest: **if Python can do it, W++ can do it**. Anything you find in a Python tutorial works here, with nineteen words swapped.

23. Cheat sheet

OUTPUT AND INPUT

```
yap("hi")
yap(a, b, sep="-")
yap("no newline", end="")
name = dm("Who? ")
n = int(dm("Number: "))
```

TRUTH

```
nocap  cap  npc
== != < > <= >=
and or not in is
```

COLLECTIONS

```
s = squad([1, 2, 3])
t = tea(a=1, b=2)
c = cult([1, 2, 2])
bodycount(s)
s[0]  s[-1]  s[1:3]
s.append(4)  s.pop()
t["a"]  t.get("z", 0)
t.items()  t.keys()
c.add(3)  c & c2  c | c2
```

STRINGS

```
f"{name} has {n} items"
f"{value:.2f}"
"a,b".split(",")
"-".join(parts)
text.strip().upper()
text.replace("a", "b")
bodycount(text)
```

RUNNING

```
python wpp.py prog.wpp
python wpp.py --emit prog.wpp
python wpp.py --keywords
python playground/server.py
```

DECISIONS

```
bet x > 10:
    yap("big")
plotwist x > 5:
    yap("medium")
nah:
    yap("small")

label = "y" bet ok nah "n"
```

LOOPS

```
spam i in range(5):
    yap(i)

spam item in things:
    yap(item)

grind n > 0:
    n = n - 1

dip      leave the loop
skrtrt   next round
```

COMPREHENSIONS

```
[n * n spam n in range(5)]
[n spam n in xs bet n > 2]
{k: v spam k, v in pairs}
```

FUNCTIONS

```
cook add(a, b=1):
    spill a + b

cook many(*args, **kw):
    spill args, kw

f = lambda n: n * 2
```

CLASSES

```
class Dog:
    cook __init__(self, n):
        self.name = n

    cook speak(self):
        spill "woof"

rex = Dog("Rex")
yap(rex.speak())
```

ERRORS

```
try:
    risky()
except ValueError as e:
    yap("oops", e)
nah:
    yap("fine")
finally:
    yap("always")

raise ValueError("nope")
```